



I've setup chrome to launch with `chrome://inspect/#remote-debugging` on port 9222 and want Claude Code to have full access to it using Chrome DevTools "<https://github.com/ChromeDevTools/chrome-devtools-mcp>" to my dev server running on port 3000 in node.js through vite.

Yes — the clean way is to run Chrome with a separate debug profile on port 9222, then register `chrome-devtools-mcp` in Claude Code so it connects to that running browser via `--browser-url=http://127.0.0.1:9222`. That gives Claude Code access to the same live Chrome session you're using against your Vite app on port 3000.^{1_1}

What to set up

Chrome DevTools MCP is an MCP server that lets coding agents like Claude control and inspect a live Chrome browser, including navigation, console, network, screenshots, and performance tooling. The project supports connecting to an already-running Chrome instance by passing `--browser-url` pointing at the remote debugging endpoint, with `http://127.0.0.1:9222` called out as the common default.^{1_1}

Your Vite dev server on port 3000 does not need special MCP config by itself; Claude reaches it by driving Chrome to `http://127.0.0.1:3000` once the browser connection is established. The important link is between Claude Code and Chrome on 9222, not between Claude Code and Vite directly.^{1_1}

Claude Code config

For Claude Code, the README shows the CLI-based MCP install format, and the same server can be configured with extra args such as `--browser-url=http://127.0.0.1:9222`. A commonly used Claude Code command format is:^{1_1}

```
claude mcp add --transport stdio chrome-devtools -- npx -y chrome-devtools-mcp@latest --browserUrl=http://127.0.0.1:9222
```

That pattern matches the project's documented use of `npx chrome-devtools-mcp@latest` plus the `--browserUrl/--browser-url` option for attaching to a running browser.^{1_2^1_1}

If your Claude Code install prefers config files instead of the CLI, the equivalent MCP entry is:

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": [
        "-y",
        "chrome-devtools-mcp@latest",
        "--browser-url=http://127.0.0.1:9222"
      ]
    }
  }
}
```

That argument format is directly consistent with the upstream README examples.^{^1_1}

Chrome launch

If you use manual port-based connection, Chrome must be started with `--remote-debugging-port=9222`, and the README says Chrome requires a non-default `--user-data-dir` for security when remote debugging is enabled. On Windows, the documented launch pattern is:^{^1_1}

```
& "C:\Program Files\Google\Chrome\Application\chrome.exe" `
  --remote-debugging-port=9222 `
  --user-data-dir="$env:TEMP\chrome-profile-stable"
```

Using a separate profile is important because the debugging port exposes the browser to local processes that can inspect and control it. The upstream docs explicitly warn not to use this on a browser session that contains sensitive activity.^{^1_1}

Your local dev flow

Once Chrome is running on 9222 and your Vite server is running on 3000, Claude Code can open or inspect `http://127.0.0.1:3000` in that Chrome instance. From there, the MCP server exposes tools for page navigation, console inspection, network requests, screenshots, script evaluation, and Lighthouse/performance work.^{^1_1}

A practical workflow is:

- Start Vite on port 3000.^{^1_1}
- Start Chrome with remote debugging on 9222 and a separate user data dir.^{^1_1}
- Add the MCP server to Claude Code with `--browser-url=http://127.0.0.1:9222`.^{^1_2}
- In Claude Code, ask it to open `http://127.0.0.1:3000`, inspect console errors, watch network requests, or take a snapshot.^{^1_1}

Verify it works

A simple check is to open `http://127.0.0.1:9222/json/version` in any browser or with curl; the docs say you should see Chrome debugging endpoint data there, including the WebSocket debugger URL. The README also notes that if you want the raw websocket endpoint instead of `--browser-url`, you can pull `websocketDebuggerUrl` from that JSON and use `--wsEndpoint=...` instead.^{1_1}

If Claude reports it cannot connect, the most likely causes are:

- Chrome was not launched with `--remote-debugging-port=9222`.^{1_1}
- Chrome was launched without a separate `--user-data-dir`.^{1_1}
- Another normal Chrome instance was already running, so the debug launch did not start the intended profile cleanly.^{1_1}
- Claude Code was configured with the wrong flag or wrong port.^{1_1}

Best option for you

Because you already enabled `chrome://inspect/#remote-debugging` and want Claude Code to use your existing browser session, the best fit is the manual connection approach with `--browser-url=http://127.0.0.1:9222`. The newer `--autoConnect` flow exists for Chrome 144+ and user-approved live attachment, but the port-based method is still the more explicit setup when you want deterministic local wiring to a dev browser session.^{1_4}

Here is the shortest end-to-end setup for Windows:

```
# 1) Run your Vite app
npm run dev
# expect http://127.0.0.1:3000 or http://localhost:3000
```

```
# 2) Launch a dedicated Chrome debug session
& "C:\Program Files\Google\Chrome\Application\chrome.exe" `
  --remote-debugging-port=9222 `
  --user-data-dir="$env:TEMP\chrome-profile-stable"
```

```
# 3) Register MCP in Claude Code
claude mcp add --transport stdio chrome-devtools -- npx -y chrome-devtools-
mcp@latest --browserUrl=http://127.0.0.1:9222
```

Then tell Claude Code something like: "Open `http://127.0.0.1:3000`, inspect console and network errors, and fix any Vite runtime issues." That uses the Chrome session you exposed on 9222 and your app served on 3000.^{1_2}

If you want, I can give you the exact Claude Code config file snippet for Windows, WSL, or macOS.

my current work flow is to run this script @echo off

REM Chrome DevTools MCP - Debug Chrome Launcher REM Launches Chrome with remote debugging on port 9222 using a separate profile. REM Safe to run alongside regular Chrome.

REM Check if debug Chrome is already running on port 9222 netstat -ano | findstr :9222 >nul 2>&1 if %errorlevel%==0 (echo Chrome debug already running on port 9222. exit /b 0)

start "" "C:\Program Files\Google\Chrome\Application\chrome.exe" ^ --remote-debugging-port=9222 ^ --user-data-dir="C:\Users\Soren.chrome-debug-profile" ^ --no-first-run ^ --no-default-browser-check ^ --window-size=1280,720

echo Chrome debug launched on port 9222.

and then open pnpm dev, which opens localhost:3000

Your workflow is already basically correct; the missing piece is just making Claude Code's MCP config explicitly attach to that Chrome instance on **127.0.0.1:9222**, then letting Claude drive the tab at **localhost:3000**.^{^2_1}

The key point is that **pnpm dev** opening **localhost:3000** is fine on its own, because Chrome DevTools MCP connects to Chrome on port 9222, and then Chrome navigates to your Vite app on port 3000. You do not need a special Vite-side bridge as long as the page is reachable in the same Chrome session.^{^2_1}

Your script is good

Your batch file already follows the upstream manual-connection model by launching Chrome with **--remote-debugging-port=9222** and a separate **--user-data-dir**, which the README says is required for security when remote debugging is enabled. Using a dedicated profile like **C:\Users\Soren\.chrome-debug-profile** is the right pattern if you want a persistent debug browser separate from your normal Chrome profile.^{^2_2}

The only thing I would tighten is the port check, because **netstat | findstr :9222** confirms something is listening on 9222, but it does not prove that the listener is actually Chrome's DevTools endpoint.^{^2_1} A stronger validation is to query **http://127.0.0.1:9222/json/version**, because the Chrome DevTools docs say that endpoint should expose Chrome's debugging metadata, including the browser WebSocket debugger URL.^{^2_1}

Best Claude Code setup

For your setup, the best Claude Code registration is the manual connection mode using **--browser-url=http://127.0.0.1:9222**, because the README documents that exact pattern for attaching the MCP server to an already-running Chrome instance. That keeps Claude attached to the same browser window you launched from your batch script, which is ideal when you want shared cookies, shared login state, and live manual-plus-agent debugging.^{^2_1}

Use this:

```
claude mcp add chrome-devtools --scope user -- npx -y chrome-devtools-mcp@latest -
-browser-url=http://127.0.0.1:9222
```

That matches the project's documented `--browser-url` option and Claude Code's CLI-based MCP installation flow. If you already added a plain `chrome-devtools` server before, remove it first or overwrite it so Claude does not keep launching its own isolated browser instead of attaching to yours. ^{2_1}

Recommended flow

Your practical workflow should look like this:

1. Run your Chrome launcher batch file so the debug Chrome profile starts on port 9222. ^{2_1}
2. Run `pnpm dev` so Vite serves your app on `http://localhost:3000` or `http://127.0.0.1:3000`. ^{2_1}
3. Start Claude Code with the `chrome-devtools` MCP server configured to use `--browser-url=http://127.0.0.1:9222`. ^{2_3}
4. Ask Claude to open `http://127.0.0.1:3000`, inspect console errors, inspect network requests, take snapshots, and interact with the app. ^{2_1}

That works because the MCP server gives Claude access to navigation, console inspection, screenshots, script evaluation, network inspection, and performance tooling inside the live Chrome session. ^{2_1}

Better batch file

I'd slightly upgrade your script so it verifies the actual DevTools endpoint instead of only checking the port. This version is more reliable on Windows: ^{2_1}

```
@echo off
setlocal

set CHROME_EXE=C:\Program Files\Google\Chrome\Application\chrome.exe
set CHROME_PROFILE=C:\Users\Soren\.chrome-debug-profile
set DEBUG_URL=http://127.0.0.1:9222/json/version

curl -s %DEBUG_URL% | findstr /C:"websocketDebuggerUrl" >nul 2>&1
if %errorlevel%==0 (
    echo Chrome debug already running and DevTools endpoint is available.
    exit /b 0
)

start "" "%CHROME_EXE%" ^
    --remote-debugging-port=9222 ^
    --user-data-dir="%CHROME_PROFILE%" ^
    --no-first-run ^
    --no-default-browser-check ^
    --window-size=1280,720

echo Waiting for Chrome DevTools endpoint...
timeout /t 2 /nobreak >nul
```

```
curl -s %DEBUG_URL% | findstr /C:"websocketDebuggerUrl" >nul 2>&1
if %errorlevel%==0 (
    echo Chrome debug launched successfully on port 9222.
    exit /b 0
)

echo Chrome started, but DevTools endpoint was not detected yet.
echo Try opening http://127.0.0.1:9222/json/version manually.
exit /b 1
```

This aligns with the documented recommendation to use `/json/version` to confirm the debuggable Chrome instance is really available.^{^2_1}

What Claude should do next

Once the MCP server is connected, Claude can work against your existing Vite app session rather than a separate hidden browser. A good first prompt is: "Open `http://127.0.0.1:3000`, check console messages, inspect failed network requests, and fix any runtime issues."^{^2_1}

If you want Claude to always use your already-open page, open the app in that debug Chrome profile first, then tell Claude to list pages and select the Vite tab before debugging. The MCP server supports page listing, page selection, navigation, console log access, screenshots, and network request inspection for exactly that use case.^{^2_1}

Two useful checks

Run these when debugging connection issues:

```
curl http://127.0.0.1:9222/json/version
```

That should return JSON from Chrome's remote debugging endpoint if the browser is available.^{^2_3}

```
claude mcp list
```

Claude Code should show the `chrome-devtools` MCP entry as installed, and if connected correctly it should be using the `npx ... chrome-devtools-mcp@latest --browser-url=http://127.0.0.1:9222` command pattern.^{^2_4}

One caveat

Keep that debug Chrome profile dedicated to development, because the README explicitly warns that any local application can connect to the remote debugging port and inspect or control that browser while the port is open. So your current separate-profile approach is not just convenient, it is the safer way to do this.^{^2_1}

If you want, I can turn your current setup into a polished Windows workflow with:

- one `start-debug-chrome.bat`
- one `start-dev-with-claude.bat`
- the exact Claude Code config or CLI commands for your machine

**

THIS IS MY MCP.JSON CONFIG "chrome-devtools": {

```
"command": "npx",    "args": [        "-y",        "chrome-devtools-mcp@latest",        "--browserUrl",  
"http://127.0.0.1:9222"    ],    "env": {        "SystemRoot": "C:\\Windows",        "PROGRAMFILES":  
"C:\\Program Files"    }    },
```

Your config is very close, and it should work in principle because the upstream docs explicitly support both `--browserUrl` and `--browser-url` for connecting to a running Chrome instance like `http://127.0.0.1:9222`. So the core issue is probably not the concept of your config, but either argument formatting, the MCP client's expected JSON shape, or whether Claude Code is actually loading this exact file.^{^3_1}

What looks right

This part is valid in spirit:

```
"command": "npx",  
"args": [  
    "-y",  
    "chrome-devtools-mcp@latest",  
    "--browserUrl", "http://127.0.0.1:9222"  
]
```

The docs show that `chrome-devtools-mcp` accepts `--browserUrl` or `--browser-url` and that the target should be the running Chrome debug endpoint URL. Your URL `http://127.0.0.1:9222` matches the documented pattern exactly.^{^3_1}

What I would change

I would switch your args to the single-token form with an equals sign:

```
"args": [  
    "-y",  
    "chrome-devtools-mcp@latest",  
    "--browser-url=http://127.0.0.1:9222"  
]
```

The upstream README includes the config example in exactly that `--browser-url=http://127.0.0.1:9222` format, which removes any ambiguity about how the MCP client passes split

arguments through to `npx`. While `--browserUrl`, `"http://127.0.0.1:9222"` may still parse correctly, the single combined arg is the safest and most canonical version. [web:1]web:5

Best final config

If your file is an `mcp.json`-style file with a top-level `mcpServers` object, I would use this exact structure:

```
{
  "mcpServers": {
    "chrome-devtools": {
      "command": "npx",
      "args": [
        "-y",
        "chrome-devtools-mcp@latest",
        "--browser-url=http://127.0.0.1:9222"
      ],
      "env": {
        "SystemRoot": "C:\\Windows",
        "PROGRAMFILES": "C:\\Program Files"
      }
    }
  }
}
```

That matches the documented config pattern for Chrome DevTools MCP with a running browser URL. If your Claude Code flavor expects `servers` instead of `mcpServers`, then the same inner block is fine but must be nested under the key your client actually reads. ^3_3^3_1

Two likely problems

The most common real-world problem is that the config is syntactically fine but Claude Code is not reading that file at all, or is reading a different scope such as user config vs project config. The second common problem is that Chrome is listening on 9222, but the endpoint is not actually reachable as a DevTools browser endpoint, which you can verify by opening `http://127.0.0.1:9222/json/version` and checking for `websocketDebuggerUrl`. ^3_5^3_4

Quick verification

Check these three things:

- Open `http://127.0.0.1:9222/json/version`; if it returns JSON with `websocketDebuggerUrl`, Chrome remote debugging is alive. ^3_2
- Run `claude mcp list`; the installed `chrome-devtools` entry should show the browser-url argument you expect. ^3_2
- Make sure there is only one active `chrome-devtools` server definition, so Claude does not use an older conflicting config. ^3_5

My recommendation

Use this exact args array:

```
"args": [  
  "-y",  
  "chrome-devtools-mcp@latest",  
  "--browser-url=http://127.0.0.1:9222"  
]
```

That is the cleanest version and the closest match to upstream documentation. I would keep your env block only if Claude Code on your Windows setup actually needs it, because the docs do not require those variables for normal operation.^{^3_3}

If you want, I can now give you:

- the exact corrected full `mcp.json`
- a matching Windows batch file pair
- a 60-second test checklist to confirm Claude is truly attached to your live Chrome session

**

is there a way to clean up your prompts for claude code?

Yes. There are a few layers: clearing the current context, starting cleaner sessions, and structuring prompts so they stay readable over time.^{^4_1}

Clear Claude Code's current context

Inside a Claude Code session you can wipe the active context buffer so new prompts aren't polluted by old ones.^{^4_2}

- Use `/clear` in the Claude Code input to reset the current context window for that session.^{^4_3^4_2}
- After `/clear`, previous conversation files still exist on disk, but the model no longer "sees" that context unless you explicitly load it again.^{^4_4}

If you're working CLI-only, many people also just start a new session instead of clearing: run `claude` again without `--resume` or `--continue` to get a fresh context.^{^4_5}

Keep sessions focused

By default Claude Code doesn't share context between separate sessions, so starting a new one is an easy way to "clean up" prompts.^{^4_4}

- When you switch to a different feature or project, start a new Claude Code session instead of continuing a huge mixed history.^{^4_4}
- Use `claude --resume` or `claude -r <id>` only when you explicitly want to bring back older context.^{^4_5}

This keeps each session's prompt history tight and avoids massive, messy transcripts. [^4_4](#)

Make prompts themselves cleaner

For the prompt text you write, a few habits help a lot over time:

- Use a short "task header" at the top when you start a session: what repo, what feature, what stack.
- Refer to files/paths directly (e.g. `src/components/Foo.vue`) and keep one main question per message.
- When you're done with a chunk of work, briefly summarize what you did in a final message; next time you can paste that summary instead of the whole old conversation.

If you'd like, tell me whether you're using Claude Code mostly via the CLI, VS Code extension, or the desktop app, and I can give you a concrete "prompt hygiene" routine tailored to that setup.

**

I want to use a pencil.pen file to apply a very specific style to my existing codebase UI. my codebase is writtern in vuetify 3

Yes — a `.pen` file can be a good way to impose a very specific visual style on an existing Vuetify 3 codebase, especially because Pencil supports a two-way workflow: importing existing code into a `.pen` design file, editing the design, and then asking the agent to apply those design changes back into code. Since your app already uses Vue 3 + Vite + Vuetify 3, the practical goal should be to use Pencil mostly for design direction, token definition, and component-level visual refinement rather than expecting a blind full-app export to cleanly replace a mature Vuetify codebase. [^5_1](#)

Best way to use it

Pencil's docs explicitly describe the strongest workflow for existing apps as: start from code, import components into the `.pen` file, make design improvements in Pencil, then update the code and iterate. That is a much better fit for Vuetify 3 than trying to generate an entirely new UI from scratch, because Vuetify already gives you structure, states, accessibility, and component behavior that you don't want to accidentally throw away. [^5_2](#)

For your stack, think of the `.pen` file as a visual source of truth for:

- spacing rhythm
- typography scale
- color tokens
- card, dialog, button, table, and form styling
- page composition patterns for owner/admin screens [^5_2](#)

Where it fits in Vuetify 3

Vuetify 3 is themeable, but "very specific style" usually requires more than just changing the theme primary color. A strong implementation usually combines Vuetify theme tokens, global CSS variables, utility classes,

and selective component overrides for things like `v-card`, `v-btn`, `v-text-field`, `v-data-table`, `v-navigation-drawer`, and app-bar styling.

Pencil docs also mention syncing variables between design and code by importing CSS variables into Pencil and exporting Pencil variables back to CSS. That makes it especially useful if you create a real token layer in your app, for example `src/styles/tokens.scss` or `src/styles/theme.css`, and then use that token layer to feed both Vuetify theme config and your custom component styling.^{^5_2}

Recommended workflow

I'd use this sequence:

1. Pick 3 to 5 high-value screens, not the whole app at once, such as dashboard, booking form, property detail, calendar view, and settings.
2. Create a `design.pen` file in the repo root so Pencil and the agent can access both the design file and your Vue/Vuetify code together.^{^5_2}
3. Import existing components or screens into Pencil first, because Pencil docs specifically support code-to-design import when the `.pen` file is in the same workspace.^{^5_2}
4. Redesign those screens in Pencil with explicit constraints like "keep Vuetify 3 component structure, change only visual system and layout density."^{^5_2}
5. Sync design tokens back into CSS variables, then map them into your Vuetify theme and scoped overrides.^{^5_2}
6. Apply changes incrementally to shared building blocks before touching every feature screen.^{^5_2}

That approach is safer and more maintainable than letting AI rewrite lots of template markup across a working app.^{^5_2}

What to ask Pencil or Claude

Pencil supports prompts like importing a component from a code path into the design, generating framework-specific code, and syncing design tokens to CSS. For Vuetify 3, your prompts should be more constrained than the generic examples in the docs.^{^5_2}

Good prompt style:

```
Recreate the booking form from src/views/bookings/BookingForm.vue in this .pen file.  
Keep the current Vuetify 3 component structure.  
Apply a more premium, editorial, low-density visual style with softer surfaces,  
stronger type hierarchy, and better spacing.  
Do not replace Vuetify components with custom raw HTML unless necessary.
```

Then after editing the design:

```
Apply this .pen style back to the existing Vuetify 3 implementation.  
Prefer changes in theme tokens, CSS variables, global styles, and lightweight  
slot/class overrides.  
Avoid breaking form validation, events, and Pinia bindings.
```

Those constraints matter because otherwise the AI may generate Vue code that looks good visually but fights Vuetify's patterns.^{^5_2}

What to change in code

For a Vuetify 3 codebase, the most durable implementation is usually:

- `theme.ts` or Vuetify setup: colors, elevation philosophy, rounded values, dark/light scheme hooks
- `src/styles/tokens.scss`: custom CSS variables for spacing, surface tones, radii, typography, shadows
- `src/styles/components/*.scss`: targeted overrides for Vuetify classes
- wrapper components for repeated patterns like page headers, cards, filters, stats, tables, dialogs

That gives you one place to apply the Pencil-defined style across the app instead of hand-restyling every screen individually.^{^5_2}

What Pencil is best at here

Pencil's product page emphasizes that design files live in the repo, can be versioned with git, and are intended to keep design and code aligned inside the same workspace. That makes it especially attractive for your workflow because you already live in VS Code and use Claude Code around an active frontend dev loop.^{^5_1}

In your case, the best use is not "replace Vuetify with generated code," but "use `.pen` as a visual spec and token source that Claude can read while editing your existing Vue/Vuetify components."^{^5_2}

My recommendation

Yes, use a `.pen` file — but use it as a controlled design layer over your Vuetify 3 system. Start with one shared surface-heavy screen and one form-heavy screen, define tokens first, then apply the style through Vuetify theme config plus targeted overrides rather than wholesale component rewrites.^{^5_2}

If you want, I can help you build the exact workflow for your repo, including:

- where to place `design.pen`
- how to structure a Vuetify 3 token file
- the best prompts to import an existing screen and push the style back into code

What single screen in your app do you want to style first?

**

owner/overview i've also got claude uusing the chrome dev tools through remote debugging so it has

Tool	Description

list_pages	List all open pages in the browser
select_page	Select a page as context for future tool calls
new_page	Open a new tab and load a URL
close_page	Close a page by its index
navigate_page	Navigate to URL, back, forward, or reload
resize_page	Resize the page window dimensions

Interaction

Tool	Description
click	Click on an element (supports double-click)
hover	Hover over an element
fill	Type into input/textarea or select from ▼